

GNN Pipeline v4 — Full Working Process Report

RNA → Protein Spatial Prediction | Team 37 (IRBM) | MultiScaleSpatialGNN, v4_final

1. Overview & Objective

This report documents the complete, current working process of the v4 GNN pipeline (`gnn_pipeline_v4_final.py`), which predicts spatial protein abundance (44 CODEX protein markers) from spatially resolved RNA expression data. The model treats each spatial bin as a graph node and uses two parallel graph neural network branches — a fine-grained local neighbourhood and a broader regional neighbourhood — to combine each bin's own transcriptomic signal with spatial context from its neighbours before predicting protein levels.

The pipeline covers the full path from raw `.h5ad` files to a submittable prediction CSV: RNA/protein normalisation, feature construction (transcriptome-wide SVD, marker-gene proxies, H&E histology features), spatial graph construction, model training (three distinct modes are supported — deployment, temporary held-out reproduction, and honest 5-fold cross-validation), and inference with exact inverse-transformation back to raw CODEX protein scale.

2. Data

Inputs are `AnnData` (`.h5ad`) objects aligned by spatial bin:

- `train_rna.h5ad` — 166,186 bins × 18,085 genes (raw counts), plus obs columns `array_row/array_col` (grid coordinates) and `pxl_row_in_fullres/pxl_col_in_fullres` (full-resolution image pixel coordinates).
- `train_pro.h5ad` — 166,186 bins × 44 proteins (raw CODEX intensities), row-aligned with `train_rna.h5ad` by construction (no explicit ID-based re-matching is performed — the pipeline trusts this alignment).
- `valid_rna.h5ad` / `test_rna.h5ad` — unlabelled RNA-only files used at inference time to produce submission predictions.

No bin-level filtering is applied anywhere in the pipeline — every bin present in the input files is used. There is no minimum-count, minimum-gene, or tissue-mask quality-control step.

3. Preprocessing & Normalisation

3.1 RNA preprocessing

Applied identically in every training/inference path:

- `scanpy.pp.normalize_total(target_sum=1e4)` — library-size normalisation; scales each bin's total counts to 10,000.
- `scanpy.pp.log1p()` — $\log(1+x)$ variance-stabilising transform.
- All 18,085 genes are retained (no HVG filtering) when `--n_hvg ≤ 0`, which is the setting used for the best-performing runs. This avoids fitting any global gene-selection step across bins that later become a held-out test set in CV.

3.2 Protein target normalisation

Two target transforms are supported, selected via `--target_transform`:

- CLR (centered log-ratio) — default / matches the team's shared reference preprocessing. $\log(x+1)$ per protein, then subtract each bin's row-wise geometric mean. Not exactly invertible: the per-bin geometric mean used to centre it is not retained, so CLR-space predictions cannot be converted back to raw protein scale.
- $\text{arcsinh}(x/150)$ — used for deployment/submission runs. Fully invertible: $\text{raw} = 150 \cdot \sinh(z)$. Standard transform for CODEX/CyTOF-style protein intensity data (stabilises variance, compresses right-skew) and is the transform that allows predictions to be converted back to real, submittable CODEX-scale values.

Whichever transform is used, the result is z-scored per protein (mean/std computed on the training bins only, saved to `preprocessing_stats.npz`) before being used as the model's regression target.

4. Feature Extraction

Each bin's node-feature vector is the concatenation of three independently constructed sources:

4.1 Transcriptome-wide SVD (384 dimensions)

The full log-normalised RNA expression matrix (all 18,085 genes) is compressed via TruncatedSVD to 384 components, fit once on the training bins (or on the fold's training bins only, inside CV) and re-applied by transform only — never refit — to validation/test/held-out data. This is an unsupervised, linear dimensionality reduction capturing the dominant transcriptome-wide co-expression structure.

4.2 Marker-gene features (42 of 44 proteins, direct gene→protein proxies)

`marker_meta.json` hand-pairs 42 of the 44 target proteins to a specific coding gene believed to encode that protein (e.g. CD8 ↔ CD8A, Ki67 ↔ MKI67, PD-1 ↔ PDCD1). For each of those genes, that gene's own normalised expression value ($\log_{10}(\text{raw} / \text{library_size} \times 1e4)$, using the full-panel library size) is pulled directly as a feature — giving the model an uncompressed, interpretable read of “how much does this bin express the gene that encodes this exact target protein,” on top of the compressed whole-transcriptome SVD signal.

Verified independently against the raw data (not taken on trust): HLA-A and HLA-DR are the two proteins with no marker-gene feature. This was checked directly against `train_rna.h5ad`'s real 18,085-gene panel — none of HLA-A, HLA-B, HLA-C, HLA-DRA, or HLA-DRB1 exist in this panel at all, so no 1:1 gene proxy is possible for either protein with this data. The original marker-building script did attempt both (they appear in its candidate-gene dictionary) and silently dropped them when the genes weren't found — this is a genuine data constraint, not an oversight.

New finding from this investigation: B2M (beta-2-microglobulin, the invariant light chain required for every HLA class I molecule's surface expression) is present in the RNA panel and is not currently used. A v2 marker set (`marker_features_v2.npy` / `marker_meta_v2.json`, new files, originals untouched) was built adding B2M as HLA-A's proxy gene, resolving 43/44 markers. HLA-DR remains unresolved — no viable candidate gene exists in this panel. Empirical check: B2M's raw correlation with HLA-A protein (arcsinh -transformed) is weak (Pearson $r = -0.049$), but this is consistent with — not worse than — every other “clean” 1:1 marker pairing already in production use (e.g. CD8A vs CD8: $r=0.005$; MS4A1 vs CD20: $r=0.005$; MPO vs MPO: $r=0.004$). Single-gene marker features carry very little standalone signal in this dataset generally; their contribution, like B2M's, would only be assessable through an actual training run, not raw correlation alone.

4.3 H&E histology features (11 dimensions, image-derived, RNA-independent)

For each bin's full-resolution pixel coordinate, a 48×48px patch is cropped from the paired H&E tissue image and reduced to 11 features: RGB mean + std (6 dims), hematoxylin/eosin stain-separated mean + std via

optical-density colour deconvolution (4 dims, using the standard Ruifrok & Johnston deconvolution matrix), and one local-texture measure (std of the Laplacian). This is the only feature source independent of the RNA data — it captures tissue morphology directly from the microscopy image.

4.4 Feature assembly

All three sources are z-scored (mean/std fit on training bins, reused at inference) and concatenated: SVD(384) + markers(42) + H&E(11) = 437 input dimensions in the baseline configuration. Markers and H&E gracefully degrade to zero-width (skipped) if their source .npy files aren't found, so the pipeline still runs without them, just with reduced feature richness.

5. Spatial Graph Construction

Two edge sets are built from each bin's (array_row, array_col) grid coordinates:

- Local k-NN graph (k=8) — each bin connected to its 8 nearest spatial neighbours; undirected.
- Regional radius graph (radius=3.5, or auto-set to 3× the median k-NN distance) — connects all bins within a distance threshold, capturing broader, density-varying neighbourhoods that a fixed-k graph misses. Isolated bins (nothing within radius) fall back to their nearest 4 neighbours so the graph stays connected.

Both graphs span every bin passed into that training run. In the default deployment mode and the --val_split mode, this means the graph covers 100% of bins (message-passing can reach every bin's RNA-derived features, even ones whose protein labels are held out of the loss). In the --cv mode, each fold builds two completely separate graphs — one over train-only bins, one over test-only bins — so there is zero connectivity between train and test; this is what makes CV the strict, honest estimate.

6. Model Architecture — MultiScaleSpatialGNN

The model processes the two graphs through two parallel branches before fusing them:

- Input projection: Linear → LayerNorm → ReLU → Dropout, mapping the 437-dim feature vector to the hidden width.
- Local branch: n_layers stacked ResidualSAGEBlocks over the local k-NN graph. Each block = GraphSAGE convolution → LayerNorm → ReLU(h + residual skip) → Dropout.
- Regional branch: an identical stack of n_layers ResidualSAGEBlocks, run independently over the regional radius graph.
- Fusion: the two branches' final hidden states are concatenated and passed through Linear → LayerNorm → ReLU → Dropout.
- Output head: Linear → LayerNorm → ReLU → Dropout → Linear, predicting all 44 protein targets simultaneously from the fused representation.

Gradient checkpointing (torch.utils.checkpoint) is applied to every block during training only — activations are recomputed on the backward pass instead of stored, trading extra compute for substantially lower GPU memory, which is what allows deeper/wider configurations to fit on a single GPU.

7. Loss Function & Optimisation

combined_loss = $0.8 \times \text{MSE} + 0.2 \times (1 - \text{mean Pearson } r \text{ across proteins})$. MSE is weighted more heavily so the model learns correct absolute magnitude, not just correct rank ordering — a pure correlation loss would only reward getting the trend right.

Optimiser: AdamW. Learning-rate schedule: linear warmup (default 10 epochs) into cosine annealing decay. Training uses automatic mixed precision (AMP) for speed/memory, up to 1000 epochs, with early stopping (default patience=50 epochs with no improvement in the tracked validation metric).

8. Training Modes

Three distinct, independently selectable training modes exist in the current script:

8.1 Default — full-data deployment training (train_final(), no flags)

Trains on 100% of train_rna.h5ad/train_pro.h5ad — every bin's label is in the loss, nothing held out. This is the model that actually ships for inference/submission. Checkpoint selection is based on best training-fit metric (there is no held-out data in this mode to measure generalisation against — that is what the CV run is for).

8.2 --val_split X (temporary, opt-in reproduction mode)

Holds out a random X fraction of bins' protein LABELS only, within a single train_final()-style run. The graph itself still spans all bins, so message-passing can still see held-out bins' RNA features — a softer, transductive generalisation estimate, not as strict as disjoint-graph CV. This mode exists specifically to reproduce one historical result (a 90/10 split run) for direct comparison, and is not the default deployment path.

8.3 --cv (5-fold spatially-blocked cross-validation)

The strict, honest generalisation estimate. Bins are grouped into 10×10-coordinate spatial blocks (block_id = (row//10)×1,000,003 + (col//10)), then GroupKFold(n_splits=5) assigns whole blocks to folds so no block is ever split across train/test — preventing spatially-correlated bins from leaking between train and test. Each fold builds fully disjoint train-only and test-only graphs, with per-fold SVD and marker/H&E z-scoring statistics fit on that fold's training bins only (zero leakage). This is the methodology that produced the 0.6566 Pearson / 0.5625 Spearman baseline reported throughout this project.

Note: this CV code path was found to have been fully removed from the script earlier in the project (per an earlier explicit simplification request) and has since been rebuilt from an ancestor version, verified via compile checks and isolated smoke tests. The default (non-CV) deployment path was unaffected throughout.

9. Baseline Hyperparameters

Parameter	Value	Notes
n_hvg	0	all 18,085 genes used, no HVG filtering
target_transform	clr / arcsinh	clr for CV baseline; arcsinh for submittable runs
svd_dims	384	TruncatedSVD components on RNA
hidden	384	hidden width per GNN branch (baseline)
n_layers	4	ResidualSAGEBlocks per branch (baseline)
k	8	local k-NN graph neighbours
radius	3.5 (or auto)	regional radius graph; auto = 3× median k-NN distance
dropout	0.3	
lr	3e-4	AdamW, cosine schedule after warmup
weight_decay	2e-3	

Parameter	Value	Notes
warmup	10 epochs	
epochs / patience	1000 / 50	early stopping on held-out metric
loss weight w	0.8	$0.8 \times \text{MSE} + 0.2 \times (1 - \text{Pearson})$

10. Evaluation Metrics

Reported per protein and averaged across all 44: Pearson r , Spearman rank correlation, RMSE, MAE. The checkpoint-selection metric during training is $0.5 \times \text{Pearson} + 0.5 \times \text{Spearman}$ on the current run's held-out set (val split or CV test fold). Codabench's own leaderboard metric, TOP10 SCC Mean, is the mean Spearman correlation across only the best-scoring 10 of the 44 proteins, computed by Codabench against its own hidden validation set.

11. Results

11.1 5-fold CV baseline (honest, disjoint-graph estimate)

Mean Pearson $r = 0.6566$, Mean Spearman = 0.5625, Mean RMSE = 0.7451, across all 44 proteins. This is the methodology described in Section 8.3 and is the reference number used to judge any architecture or hyperparameter change.

11.2 Per-protein performance (best / weakest, from the CV baseline run)

Protein	Pearson r	Spearman
Granzyme B	0.807	0.728
FOXP3	0.805	0.768
PD-L1	0.790	0.576
PDGFR	0.789	0.630
MPO	0.787	0.348
CD68	0.782	0.510
ICOS	0.780	0.669
FIBR	0.770	0.609
TOX	0.765	0.583
GFAP	0.758	0.723
...	...	...
CD31	0.533	0.502
CD16	0.514	0.495
CD8	0.490	0.568
SMA	0.456	0.431
HLA-A	0.357	0.343

Top 10 and bottom 5 of 44 proteins by Pearson r , CV baseline run. HLA-A is the single weakest-performing protein — consistent with it being one of the two proteins lacking a direct marker-gene feature (Section 4.2).

11.3 90/10-split reproduction (submittable, CODEX-scale)

A historical 90/10 held-out run (val_split mode, Section 8.2) was reproduced with the arcsinh target transform (needed for exact CODEX-scale invertibility). Result: val Pearson = 0.687, val Spearman = 0.622, close to the original historical run (0.7248 / 0.6237 with CLR) — Spearman nearly identical, Pearson somewhat lower, most likely attributable to the CLR→arcsinh target-encoding change rather than a methodology difference.

11.4 Current submission status

The GNN's most recent Codabench submission (the 90/10-split reproduction, Section 11.3) scored 0.59 on TOP10 SCC Mean (Codabench's own metric: mean Spearman correlation across the best-scoring 10 of 44 proteins, computed against Codabench's hidden validation set).

Worth noting: the GNN's own top-10-protein CV subset (computed locally from Section 11.2's data) implies a substantially higher ceiling (~0.636 Spearman) than the currently submitted score — suggesting the deployed/submitted model under-represents what the architecture can do, since that submission used the softer val_split methodology (Section 8.2) rather than a properly-trained full-data final model matched to the CV-validated configuration (Section 8.1/8.3).

12. Deployment Process & Code Components

The pipeline is split across several files with distinct responsibilities, orchestrated as a local-Mac-to-EC2-GPU workflow. This section walks through every component and what it actually contributes to getting from raw data to a submitted prediction.

12.1 gnn_pipeline_v4_final.py — the core training script

This is the single script that implements everything in Sections 3–10 above. It is invoked with a CLI flag that selects which of the three training modes (Section 8) to run, and is the only script that ever trains the model. Its internal structure:

- preprocess() — loads train_rna.h5ad/train_pro.h5ad once, applies RNA normalisation (Section 3.1), optionally caches the result to disk so repeated runs skip re-loading. Called at the start of every mode.
- make_cv_splits() — builds the 10×10 spatially-blocked GroupKFold split described in Section 8.3, or reuses an existing cv_splits.json if one is already present in the output directory (or an externally supplied one, via --cv_split_path). Only invoked when --cv is passed.
- train_final() — implements both the default 100%-data deployment mode (Section 8.1) and the --val_split reproduction mode (Section 8.2), since they share almost identical logic; the only difference is whether a held-out label slice is carved out before the training loop starts. This is the function that writes the final_model/ artifact directory.
- run_cv() — implements the 5-fold disjoint-graph CV described in Section 8.3: loops over folds, builds two separate graphs per fold, fits SVD/marker/H&E scaling per-fold-train-only, trains a fresh model per fold, and aggregates per-protein metrics across all 5 at the end. Only invoked when --cv is passed.
- MultiScaleSpatialGNN, ResidualSAGEBlock, combined_loss, build_knn_graph, build_radius_graph, mean_pearson_r, mean_spearman_r, per_protein_metrics — shared building blocks (Sections 5–7, 10) used identically by both train_final() and run_cv(), so the model architecture and loss are guaranteed identical across every training mode; only the data split differs.
- main() — the CLI entry point; parses arguments (target transform, architecture hyperparameters, --cv/--val_split/--cv_split_path/--max_folds flags) and dispatches to train_final() or run_cv().

12.2 infer_v4_final.py — turning a trained model into predictions

A separate script, run after training completes, that loads a `final_model/` directory produced by `train_final()` and applies it to new, unlabelled RNA data (`valid_rna.h5ad` or `test_rna.h5ad`). Its contribution is guaranteeing that inference-time preprocessing is bit-for-bit identical to training-time preprocessing — it never refits anything, only re-applies saved artifacts (Section 12.3). This is the script whose output CSV is what actually gets packaged for Codabench submission (Section 11.3/11.4 both came from this script's output).

12.3 `shared_preprocessing/` — feature-building scripts

Three standalone scripts build the auxiliary feature sources described in Section 4, each run once (not per training run) and cached to disk:

- `build_marker_features.py` — one-off script that reads `train_rna.h5ad` directly, resolves each of the 44 target proteins to a candidate marker gene (silently dropping any that aren't present in the panel), and writes `marker_features.npy` + `marker_meta.json`. This is what produced the original 42-of-44 marker set (Section 4.2).
- `build_marker_features_for.py` — a generalised version of the above that reuses an existing `marker_meta.json` (rather than deriving its own gene list) so it can be pointed at any RNA file — this is what `infer_v4_final.py` calls automatically if a dataset's marker features haven't been precomputed, guaranteeing the valid/test marker features use the exact same gene list as training.
- `extract_he_features.py` (run via `run_extract_he_features.command`) — one-off script that reads the full-resolution H&E tissue image and produces `he_features.npy` (Section 4.3). This is the slowest of the three (image I/O + per-bin patch cropping across ~166k bins) and is why it is precomputed once rather than run inline during training.
- `build_marker_features_v2.py` — new script from this investigation, adds a B2M-based proxy for HLA-A (Section 4.2 finding) without touching the original `marker_features.npy`/`marker_meta.json`, producing `marker_features_v2.npy`/`marker_meta_v2.json` as a separate, opt-in feature set.

13. Known Issues, Fixes & Ongoing Work

- CV code path removal & rebuild: the 5-fold CV path was found to have been entirely removed from the script at an earlier project stage; several “CV” experiment runs were silently just running full-data training with no held-out fold at all. Rebuilt as an explicit, opt-in `--cv` flag reproducing the original disjoint-graph methodology exactly; verified via compile checks and isolated smoke tests.
- Shared CV split standardisation: adopted the team's shared `cv_splits.json` (from the `preprocessing_cv_emily` repo folder) via a new `--cv_split_path` option, so results stay comparable to any teammate's model using the same split rather than merely being internally consistent. A second, stricter team split (`cv_splits_patches.json`, KMeans-patch + buffer-zone based) exists but is not currently used — confirmation of which is the team's intended standard is still open.
- Marker-gene coverage gap: HLA-A and HLA-DR verified to have no viable marker-gene proxy in this RNA panel. A B2M-based proxy for HLA-A was built and empirically checked (Section 4.2) — biologically justified, but not yet confirmed to improve model performance in an actual training run.
- Depth-scaling experiment (`n_layers=6`): in progress. `hidden=384` (matching baseline) OOM'd on the available 16GB T4 GPU; currently running with `hidden` reduced to 256 as a workaround, which means results won't be a clean single-variable comparison against the `n_layers=4`/`hidden=384` baseline. A clean version would require a larger-VRAM GPU instance.
- Submission gap: the CV-implied ceiling for the GNN's top-10 proteins (~0.636 Spearman) is meaningfully higher than the currently-submitted leaderboard score (0.59) - the recommended next step is training and

submitting a proper full-data final model matching the CV-validated configuration (rather than the softer `val_split` reproduction), and/or ensembling the 5 CV fold-models' predictions at inference time.

14. Summary

The v4 pipeline is a spatially-aware, multi-branch GNN that fuses transcriptome-wide (SVD), targeted marker-gene, and histology-derived features over two complementary spatial graphs to predict 44 protein targets from RNA data. Its honestly-validated (5-fold, disjoint-graph) generalisation estimate is 0.6566 Pearson / 0.5625 Spearman across all proteins, with a top-10-protein subset around 0.78 Pearson / 0.64 Spearman. Deployment runs entirely on a remote EC2 GPU instance via matched `run/poll` .command scripts, with every step of the artifact chain (training → `final_model/` → inference → submission CSV) traceable back to a specific script invocation. The currently deployed Codabench submission does not yet fully reflect the CV-validated ceiling; the clearest near-term levers are (1) submitting a properly CV-matched full-data final model rather than the current `val_split`-based one, (2) fold-ensembling, and (3) confirming whether deeper architectures (`n_layers=6`, pending a clean bigger-GPU test) meaningfully improve on the baseline.

